Daffodil International University
Department of Software Engineering

System Design Document

Unit 3: Auction Engine; Listings, Search & Fixed-Price

TrustBazaar: A Verified Peer-to-Peer Auction and Marketplace Platform for Used Goods

S. M. Nihal Ahmed (232-35-002)
Afrim Hossen Khan (232-35-076)
Sabikun Nahar Sinthia (232-35-475)

Status	Draft for team review
Version	0.1, 14 October 2026
Implements	SRS v2.0 (`docs/srs.tex`)
Depends on	Unit 1 (Identity & Access), Unit 2 (Order, Payment & Escrow)
Database schema	`docs/design/sql/03_listings_auctions.sql`, tested on PostgreSQL (67 checks and concurrency tests, Appendix B)

Contents

1 Introduction **3**
1.1 Purpose . . . 3
1.2 Scope . . . 3
1.3 Requirements covered . . . 3

2 Unit Overview **4**
2.1 Responsibilities . . . 4
2.2 Components . . . 4
2.3 Why the bid decision runs in the database . . . 5

3 Interfaces **5**
3.1 HTTP API . . . 5
3.2 Live channel . . . 6
3.3 Ports provided . . . 7
3.4 Events . . . 7
3.5 Ports required . . . 7

4 Data Design **8**
4.1 Tables and rules the database enforces . . . 8

5 The Bid Engine **9**
5.1 Rules . . . 9
5.2 Time, order and the end of an auction . . . 9
5.3 Placing a bid . . . 10
5.4 Voids and recomputation . . . 11
5.5 Closing and second chance . . . 11
5.6 Shill-bidding flags . . . 11

6 Listings **12**
6.1 Life cycle . . . 12
6.2 Photos . . . 13
6.3 AI-drafted text . . . 13

7 Search **13**
7.1 Text . . . 13
7.2 Filters, distance and sorting . . . 13
7.3 Featured listings and recommendations . . . 14
7.4 Performance . . . 14

8 Performance and Scaling **14**

9 Security Design **14**

10 Parameters **15**

11 Testing **15**

12 Notes for Integration **16**

A Traceability **16**

B Database Schema and Tests **17**

B.1 Test results 17

B.2 Schema 19

1 Introduction

1.1 Purpose

This document is the third unit of TrustBazaar's System Design Document. It covers two closely linked parts of the SRS:

- **listings**: creating and managing listings, photos, categories, screening, search, the featured label, and the stock side of fixed-price purchase;
- **the auction engine**: server-authoritative bidding with proxy maximums, reserve prices and anti-sniping, the live channel to viewers, closing, second-chance offers and non-payment strikes.

Acceptance criterion 1 (two simultaneous bids never both win) and the stock half of criterion 2 (no over-sale) depend on this unit.

The unit follows conventions DC-1 to DC-13 (Unit 1 Section 2; Unit 2 Section 2.3) and builds on both earlier units. It uses Unit 1 for identity checks, rate limits, audit and staff actions. It creates auction orders through Unit 2's `OrdersPort` and provides the stock port that Unit 2's checkout uses.

1.2 Scope

In scope:

- the listing life cycle, photos and the media pipeline, categories and prohibited items, keyword screening, clarifications and listing limits;
- visibility of listings when a seller is unverified or suspended;
- the stock port: reserve, release and mark sold;
- search and filters, distance, sorting and the featured strip;
- the auction rules and the bid engine, bid history and bidder masks, the live channel, closing;
- second-chance offers, strikes and bidding restrictions, voiding and recomputing bids, and shill-bidding flags.

Out of scope, owned by later units:

- the game-account details and ownership proof (Unit 5, which approves game-account listings through this unit's port);
- the delivery of notifications, including outbid notices outside the live channel (Unit 6);
- the operator queues and their screens for reviewing listings, reports and flags (Unit 7);
- payment of featured fees (Unit 2).

1.3 Requirements covered

- **Functional:** FR-LST-01 to FR-LST-13; FR-SRC-01 to FR-SRC-06; FR-AUC-01 to FR-AUC-18; FR-FIX-02, FR-FIX-04 and FR-FIX-05 (stock side); FR-MON-05 (featured label); FR-KYC-10 and FR-TNS-05 (hiding listings); FR-TNS-02 and FR-TNS-08 (flags).
- **Security:** SEC-ABU-01 (bid and listing limits), SEC-WEB-05 (hidden fields), SEC-DAT-08 and SEC-INP-04 (listing photos), SEC-SES-07 (live channel).
- **Other:** IF-UI-03, IF-COM-02; NFR-PERF-01 to NFR-PERF-03, NFR-PERF-05, NFR-PERF-08; NFR-REL-01, NFR-REL-03; NFR-SCL-01 to NFR-SCL-03; NFR-USE-01; DR-03, DR-04, DR-10.

2 Unit Overview

2.1 Responsibilities

1. Publish only listings that meet the rules: allowed category, photos, defect declaration and parcel details. Lock their content once anyone has bid or bought.
2. Never sell more units than exist, however many buyers arrive at once.
3. Find listings quickly in Bangla and English, by text, filters and distance, without revealing exact locations.
4. Decide every bid under one rule set, in a single serial order per auction, against the server's clock. Publish the result to every viewer within a second.
5. Close each auction exactly once, create the winner's order, and run second-chance offers and strikes when a winner does not pay.

2.2 Components

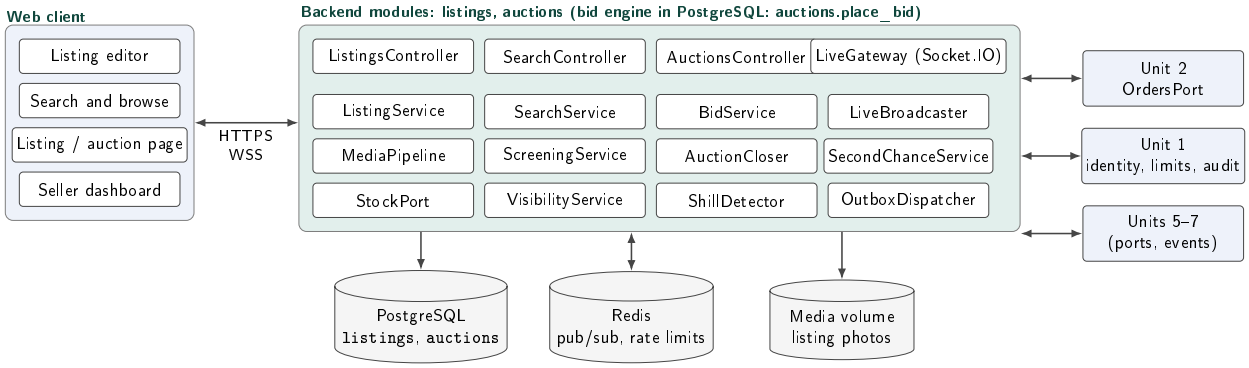


Figure 1: Components of the Auctions, Listings & Search unit. The bid engine's critical section is the database function `auctions.place_bid`; Redis carries live messages between backend instances.

Component	Responsibility
ListingService	Create, edit, publish, cancel and clarify listings; enforce the listing limit; record status history.
MediaPipeline	Check, strip, resize and store listing photos (Section 6.2).
ScreeningService	Match listing text against the screening terms; hold matches for review (FR-LST-11).
StockPort	`reserve`, `release` and `markSold` for Unit 2 (DC-13).
VisibilityService	Hides and restores a seller's listings on identity events (FR-SRC-04, FR-KYC-10, FR-TNS-05).
SearchService	Text search, filters, distance, sorting, featured strip, pagination (Section 7).
BidService	Checks eligibility and limits, calls `auctions.place_bid`, and hands the result to the broadcaster.
LiveGateway, LiveBroadcaster	Socket.IO rooms per auction; state messages with sequence numbers; outbid notices.
AuctionCloser	Closes due auctions exactly once and creates the winner's order (Section 5.5).
SecondChanceService	Strikes, restrictions and second-chance offers after an unpaid win.
ShillDetector	Scores bidding patterns and raises flags for operators (FR-AUC-18).

2.3 Why the bid decision runs in the database

The rule "each bid is evaluated against the state left by the previous one" (FR-AUC-03) requires the read–decide–write of a bid to be serialised per auction. The decision is therefore a PL/pgSQL function, `auctions.place_bid`, which locks the auction row, decides and writes in one database round trip. The service layer keeps everything that does not need the lock: authentication, the KYC gate, rate limits, and the broadcast after commit.

This has three advantages:

- **Short lock time.** The lock is held only for the time the function takes, rather than across several network round trips. Measured in the database over 500 accepted bids, a call takes about 0.19 ms, excluding network and commit time.
- **One source of truth.** There is no second lock in Redis that could disagree with the database (NFR-REL-01).
- **Testable rules.** The rules can be tested directly on the database, including under real concurrency (Appendix B.1).

The cost is that this business logic lives in SQL rather than TypeScript. To keep that manageable, the rules are a single pure function, `auctions.resolve`, used both for live bids and for replay, and the TypeScript tests call the same function.

3 Interfaces

3.1 HTTP API

All paths are under `/api/v1`; policies and rate-limit rules are those of Unit 1. Responses never include a bidder's maximum, the reserve amount, a seller's exact coordinates or any bidder's identity (SEC-WEB-05).

Method	Path	Policy	Purpose (requirement)
Listings			
`GET`	`/categories`	Public	Category tree in Bangla and English; prohibited ones shown as policy only (FR-LST-10)
`POST`	`/listings`	Verified(list)	Create a draft (FR-LST-01 to 06)
`PATCH`	`/listings/{id}`	Seller	Edit a draft, or an active listing before its first bid or order (FR-LST-07)
`POST`	`/listings/{id}/photos`	Seller	Upload a photo (Section 6.2)
`DELETE`	`/listings/{id}/photos/{pid}`	Seller	Remove a photo while editable
`POST`	`/listings/{id}/publish`	Verified(list), seller	Publish, or send for review (FR-LST-11, 12)
`POST`	`/listings/{id}/clarifications`	Seller	Add a time-stamped clarification (FR-LST-07)
`POST`	`/listings/{id}/cancel`	Seller	Cancel where allowed (FR-LST-08)
`POST`	`/listings/{id}/ai-draft`	Seller	Draft text from photos, returned for editing only (FR-LST-13, Could)
`GET`	`/listings/{id}`	Public	Listing page with rounded distance, clarifications, auction state
`GET`	`/me/listings`	Verified	My listings and their states
Search			
`GET`	`/search`	Public	Text, filters, sort, cursor (FR-SRC-01 to 05)
`GET`	`/recommendations`	SignedIn	Labelled recommendations (FR-SRC-06, Could)

Method	Path	Policy	Purpose (requirement)
Auctions			
GET	/auctions/{id}/state	Public	Price, masked leader, end time, server time, reserve status, sequence
GET	/auctions/{id}/bids	Public	Masked bid history (FR-AUC-13)
POST	/auctions/{id}/bids	Verified(bid)	Place a bid or raise one's maximum (FR-AUC-02 to 07)
GET	/auctions/{id}/my-status	Verified	My maximum, whether I lead (shown only to me)
POST	/auctions/{id}/second-chance/{oid}/decision	Verified(buy)	Accept or decline an offer (FR-AUC-15)
Staff			
POST	/staff/listings/{id}/review	Staff(operator)	Approve or return a listing in review
POST	/staff/listings/{id}/remove	Staff(operator)	Remove for a policy breach, with reason
POST	/staff/auctions/{id}/cancel	Staff(operator)	Cancel an auction with bids (FR-LST-08)
POST	/staff/bids/{bid}/void	Staff(operator)	Void a bid; recompute (FR-AUC-10)
GET	/staff/auctions/{id}/maximums	Staff(operator), open flag	Bidders' maximums, only while a fraud flag on that auction is open; audited (FR-AUC-08)
POST, DELETE	/staff/screening-terms	Staff(operator)	Maintain screening terms (FR-LST-11)

A bid request is `{ maximum }` in poisha: a plain bid is a maximum equal to the amount. The response is the bidder's own outcome (leading, outbid at the new price, or maximum raised), the new public state and its sequence number. Refusals return `422` with a stable code: `below_minimum` (with the minimum), `max_not_higher`, `auction_ended`, `auction_not_open`, `seller_cannot_bid` or `bidder_restricted`.

3.2 Live channel

The Socket.IO connection authenticated by Unit 1 (SEC-SES-07) carries:

Message	Content and use
join / leave	Client joins the room `auction:{id}` while viewing an auction page. Any signed-in user may join; the room carries public data only.
auction.state	Sent to the room after every committed change: `price`, `bidCount`, `leaderAlias`, `endAt`, `reserveMet`, `serverTime` and `seq` (the auction's `version`). Clients discard a message whose `seq` is not higher than the last one applied, so reordering in transit cannot show an older state.
auction.closed	Final status (sold or unsold) and final price.
auction.outbid	Sent only to `user:{id}` of the previous leader, with the new price (FR-AUC-12). The Notifications unit also receives the event for the notification history.

- **Countdown.** Every state message and the `/state` response carry `serverTime`. The client computes its clock offset from them and counts down to `endAt` on the server's clock (FR-AUC-01).
- **Reconnection.** On reconnecting, the client fetches `/state` again before applying further messages (IF-COM-02).
- **Guests.** The SRS puts live delivery on the authenticated connection, so guests do not join rooms. The page polls `/state` every DV-19 instead, and shows a "sign in for live updates" hint.

- **Several backend instances.** Messages pass between instances through the Socket.IO Redis adapter (NFR-SCL-01).

3.3 Ports provided

```
interface ListingsPort {                          // used by Unit 2 (DC-13)
  reserve(listingId: string, qty: number, tx: EntityManager): Promise<boolean>;  // false = not available
  release(listingId: string, qty: number, tx: EntityManager): Promise<void>;
  markSold(listingId: string, qty: number, tx: EntityManager): Promise<void>;
  snapshot(listingId: string): Promise<ListingSnapshot>;  // stored on the order (DR-03)
}
interface ListingsCommands {                      // used by Units 5 and 7
  approveForPublication(listingId: string, byStaffId: string, reason: string): Promise<void>;
  returnToDraft(listingId: string, byStaffId: string, reason: string): Promise<void>;
  remove(listingId: string, byStaffId: string, reason: string): Promise<void>;
}
interface AuctionQuery {                          // used by Units 2 and 7
  getPublicState(auctionId: string): Promise<AuctionState>;
  getFlags(auctionId: string): Promise<Flag[]>;
}
```

3.4 Events

Event	Use
Published (outbox, DC-6)	
`listings.listing.published`, `.removed`, `.sold`, `.ended_unsold`	Notifications; Unit 5 for game-account listings.
`auctions.auction.awarded`, `.unsold`, `.cancelled`	Notifications to winner, seller and bidders (FR-AUC-14, FR-LST-08).
`auctions.bid.outbid`	Notifications history (FR-NTF-01).
`auctions.offer.made`, `.expired`	Notifications (FR-AUC-15).
`auctions.bidder.restricted`	Notifications: tells the bidder why (FR-AUC-16).
`auctions.flag.raised`	Trust & Safety queue (FR-AUC-18).
Consumed (idempotent)	
`identity.kyc.revoked`, `identity.account.suspended`	Hide the seller's listings (`seller_visible = false`).
`identity.kyc.verified`, `identity.account.reinstated`	Show them again.
`orders.order.paid`	`markSold`; an auction listing becomes `sold`.
`orders.order.cancelled`	`release` for fixed-price orders.
`orders.order.payment_expired`	Strike and second-chance offer (Section 5.5).
`payments.featured.started`, `.ended`	Set or clear `featured_until`.
`payments.subscription.changed`	Listing limit (FR-LST-09, FR-MON-08).

3.5 Ports required

- **Unit 1:** `IdentityQuery.assertCanTransact` (list, bid, buy); `RateLimiter`; `AuditService`; staff policies.
- **Unit 2:** `OrdersPort.createAuctionOrder` inside the closing transaction (DC-13); `ProStatusQuery.isPro` for the listing limit.
- **Optional services** (Could Have, advisory only, behind a port and replaceable or switchable off): `VisionDraftPort`, for draft text from photos (FR-LST-13); and the recommendation scorer (FR-SRC-06).

4 Data Design

The schema is in `docs/design/sql/03_listings_auctions.sql` (Appendix B), loaded after Units 1 and 2. The unit owns two schemas: `listings` (categories, screening terms, listings, photos, clarifications, status history, the stock functions and the search view) and `auctions` (auctions, bids, bidder aliases, strikes, restrictions, second-chance offers, flags, outbox and the bid-engine functions). It also enables the `pg_trgm` extension, which is part of the standard PostgreSQL distribution.

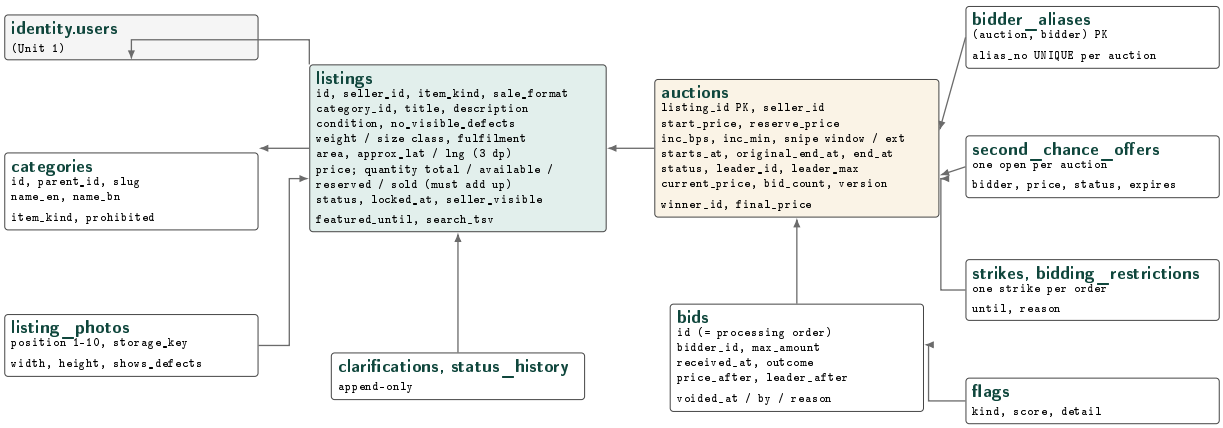


Figure 2: Listings and auctions. An auction is the auction-specific part of an auction listing (same primary key). The last arrow points to the auction, drawn through the bids box for legibility.

4.1 Tables and rules the database enforces

Table	Rules enforced by the database
`listings`	Prohibited categories refused, and category must match item kind (trigger). New listings start as drafts. Item kind and sale format fixed after publication. Content locked after the first bid or order. Only the permitted status changes (trigger). **Available + reserved + sold = total**, all non-negative. Auctions and game accounts have quantity 1. Physical items need condition, defect declaration, fulfilment and location, and courier needs parcel classes. Game accounts have none of the physical fields.
`listing_photos`	Up to 10 per listing, numbered. Publishing a physical listing needs two photos, and a defect photo or the no-defects declaration (trigger).
`clarifications`, `status_history`	Append-only.
`auctions`	Reserve $\geq$ start price. End time only moves later. Leader, maximum and price set together. Price between the start price and the leader's maximum. The seller can never lead. An award has a winner and price, and meets the reserve.
`bids`	Never deleted. Only the void fields may change, and only once (trigger).
`bidder_aliases`	One alias per bidder per auction, unique within the auction.
`strikes`	One per unpaid order.
`second_chance_offers`	One open offer per auction. A bidder is offered an item at most once.

5 The Bid Engine

5.1 Rules

Every bid is a *maximum*: a plain bid is a maximum equal to the amount, and a proxy bid states a higher one. `auctions.resolve` decides a bid from the current leader L with maximum M_L, the current price P, and a new bid from bidder B with maximum M_B. In the table, $\text{inc}(x) = \max(\lceil 2\% \cdot x \rceil_{\text{taka}}, \text{ BDT } 50)$ (CP-08).

Case	Result	Requirement
No bids yet	Needs $M_B \geq$ start price. B leads at the start price, whatever M_B.	FR-AUC-02
$B = L$	Needs $M_B > M_L$; the maximum is raised and the price does not change.	FR-AUC-06
$B \neq L$, $M_B < P + \text{inc}(P)$	Refused: `below_minimum`.	FR-AUC-02
$B \neq L$, $M_B > M_L$	B leads at $\min(M_B,\ M_L + \text{inc}(M_L))$.	FR-AUC-06
$B \neq L$, $M_B \leq M_L$	L keeps the lead at $\min(M_L,\ M_B + \text{inc}(M_B))$. Equal maximums: the earlier one wins at that maximum.	FR-AUC-06, FR-AUC-07
After any of the above	If a reserve is set, the price is below it and the leader's maximum reaches it, the price rises to the reserve.	FR-AUC-09

The last rule is necessary rather than cosmetic. Without it, a bidder whose maximum is above the reserve could still hold a *price* below it, because the proxy only bids as much as needed to beat the second bidder. The auction would then close as unsold although a willing buyer existed. With it, "reserve not met" (FR-AUC-09) means exactly that no bidder's maximum reaches the reserve.

Under these rules the price never decreases. When the auction ends, the price equals the lower of the leader's maximum and the second-highest maximum plus the increment, as FR-AUC-07 requires. Both properties are checked after the concurrency test (Appendix B.1).

Worked example (from the test script; start BDT 1,000; increment BDT 50 at these prices):

Bid	Outcome	Price after	Shown in history
Asha, max 1,500	leads (first bid)	1,000	Bidder 1: 1,000
Babu, max 1,049	refused, minimum 1,050	—	—
Babu, max 1,200	outbid by Asha's proxy	1,250	Bidder 2: 1,200
Babu, max 1,500	outbid: tie, earlier maximum wins	1,500	Bidder 2: 1,500
Chaya, max 2,000	leads	1,550	Bidder 3: 1,550
Chaya, max 2,500	raises own maximum	1,550	Bidder 3: 1,550

A leading bid is shown at the price it set, and an outbid bid at the bidder's own amount. The leader's maximum (here BDT 2,000, then 2,500) never appears (FR-AUC-08, FR-AUC-13).

5.2 Time, order and the end of an auction

- **Receipt time.** The BidService opens the database transaction as soon as the request has passed its checks, and `place_bid` takes `now()`, the transaction's start time, as the receipt time. All instances therefore use one clock, the database server's (AS-2), and the time is fixed before any waiting for the lock. A bid is accepted only if its receipt time is before `end_at` as it stands when the bid is decided (FR-AUC-04). Because extensions only ever move `end_at` later, a bid received before the end is never refused because it waited.
- **Anti-sniping.** A bid received within CP-09 of `end_at` sets `end_at` to receipt + CP-09 (FR-AUC-05). Every accepted bid in the window extends, including one immediately outbid by a proxy, because it is a valid bid.

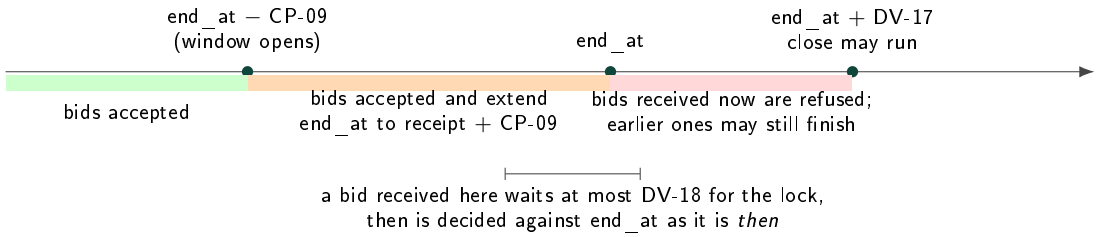


Figure 3: The end of an auction. Acceptance depends only on the receipt time; the close waits long enough for every bid received before the end to be decided.

- **Bounded wait and close grace.** Each bid waits at most DV-18 (1 second) for the auction row lock. The closer acts only after `end_at` + DV-17 (2 seconds), which is longer than DV-18. So when the close decides, every bid received before the end has already been decided or has timed out with a "try again" answer. A late extension pushes `end_at` on, and the closer, which re-reads it under the lock, simply waits for the new time.
- **Processing order.** Bids are decided one at a time in the order they obtain the lock; the bid id, assigned under the lock, records that order. Receipt order can differ, because a bid received earlier may wait longer. The concurrency tests saw between 3 and 10 such cases per run of 300 bids. Replays (for history and after a void) therefore follow bid-id order. An earlier draft replayed in receipt order, and under contention its replay disagreed with the live decisions on 11 of 31 bids. That draft was corrected, and the tests now check that replay equals the live decisions in every run.

5.3 Placing a bid

1. **BidService**, before the lock:
   - authenticates the bidder and calls `IdentityQuery.assertCanTransact(bid)`: verified, e-mail verified, not suspended;
   - consumes the bid rate limit CP-35 (FR-AUC-17);
   - checks the listing is visible.

   The proxy's automatic answers are part of a bid's own decision, so they consume no rate limit.
2. **One transaction** calls `auctions.place_bid(auction, bidder, maximum)`. Under the row lock, the function:
   - checks the status, start and end times, that the bidder is not the seller, and any restriction;
   - applies `resolve` and anti-sniping;
   - assigns the bidder's alias, records the bid and updates the auction, incrementing `version`;
   - locks the listing on the first bid (FR-LST-07).
3. **After commit**, the LiveBroadcaster:
   - emits `auction.state` to the room;
   - sends `auction.outbid` to the previous leader if the lead changed;
   - writes `auctions.bid.outbid` for the notification history.

   The response to the bidder goes out at the same time.

The broadcast is a best-effort message published after commit. A missed message is harmless, because the next message carries a higher sequence number and full state, and reconnecting clients re-fetch the state. Only the outbid event, which feeds the notification history, goes through the outbox.

5.4 Voids and recomputation

An operator voids a bid only with a reason, on evidence of fraud or account compromise (FR-AUC-10). `auctions.recompute` then replays the remaining bids in processing order through `resolve` and sets the auction to the result, as if the voided bids had never been placed. Bidders whose position changed are notified.

The end time is *not* shortened: an extension caused by a voided bid stays, because other bidders may already have acted on it. A remaining bid that would no longer be valid on replay, which is possible only after a void, is kept in the record but shown as not counted.

5.5 Closing and second chance

The AuctionCloser runs every few seconds over auctions due by `end_at` + DV-17, with `SKIP LOCKED`. For each, one transaction:

1. calls `auctions.close_if_due`, which re-checks the state under the lock and returns `awarded`, `unsold` or the existing status, so a repeated run has no effect;
2. if awarded, calls `OrdersPort.createAuctionOrder` in the same transaction (DC-13), so an award without an order, or an order without an award, cannot exist;
3. if unsold, moves the listing to `ended_unsold`;
4. writes the events and audits.

When Unit 2 reports `orders.order.payment_expired` for a winner:

- `auctions.record_strike` records the strike, at most once per order, and applies the bidding restriction once CP-12 is reached (FR-AUC-16).
- `auctions.next_second_chance` chooses the next eligible bidder. That is the highest maximum among bidders who have not won or been offered the item, whose bids are not voided and who are not restricted, with ties going to the earlier bidder. The offer is at that bidder's highest bid and expires after CP-11.
- If the offer is accepted, a `second_chance` order is created. If it is declined or expires, the next bidder is tried. When none remain, the listing ends `ended_unsold` and the seller may relist (FR-AUC-15).
- Identity eligibility (verified and not suspended) is checked again on acceptance.

5.6 Shill-bidding flags

The ShillDetector (Should, FR-AUC-18) runs after each auction closes and every few minutes on open auctions with many bids. It never blocks a bid; it raises flags with a score for the operator queue. It looks for:

- several bidders verified within the last 7 days placing small increments;
- bidders linked to the seller by device, payout account or payment instrument (from the Unit 1 and Unit 2 ports, FR-TNS-08);
- a bidder who repeatedly bids only in the last seconds across auctions;
- a bidder who raises the price against the same seller's other bidders but never wins.

While a flag on an auction is open, an operator may see its bidders' maximums; every such view is audited (FR-AUC-08).

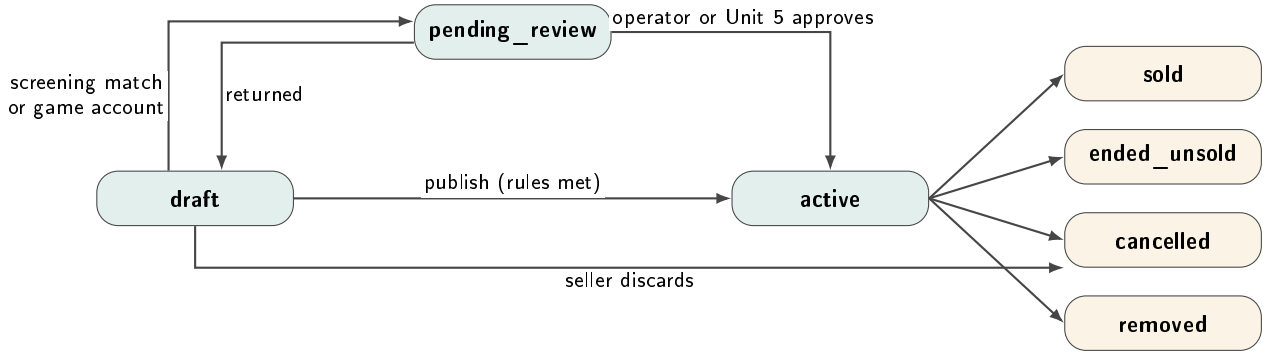


Figure 4: Listing status (FR-LST-12). Terminal states are shaded; relisting creates a new listing. An operator may also move a listing from `pending_review` to `removed` (not drawn). The trigger `listing_change` refuses every other change, and every change is written to `status_history`.

6 Listings

6.1 Life cycle

- **Create.** Requires `@Verified(list)`. A draft records title, description, category, item kind and sale format. For physical items it also records condition, the defect declaration, the fulfilment options and, for courier, the parcel classes (FR-LST-01 to FR-LST-03). The location defaults to the seller's saved area, rounded to 3 decimal places of latitude and longitude (about 110 m); only the district name is shown (FR-LST-06).
- **Publish.** The ListingService:
  - takes an advisory lock on the seller, so two parallel publishes cannot both slip under the limit, then counts the seller's active listings against CP-28 (Pro status from Unit 2);
  - runs the screening;
  - moves the listing to `active`, or to `pending_review` if a term matched or the item is a game account. Game accounts wait for Unit 5's ownership approval through `ListingsCommands`.

  The database trigger then checks the photo and declaration rules. Publishing an auction also creates its auction row, with the start, reserve and duration (CP-07) and the increment and anti-sniping parameters copied from the current parameter version (FR-OPS-08).
- **Edit and clarify** (FR-LST-07). Until the first order or bid sets `locked_at`, the seller may edit title, description and photos, and for fixed price also the price and quantity. After that the database refuses content changes, and the seller may only add clarifications, which are shown below the description with their time. An auction's start price, reserve and duration are never editable after publication.
- **Cancel** (FR-LST-08). A fixed-price listing can be cancelled while no unpaid or unfulfilled order holds a reservation, and an auction while it has no bids. An auction with bids can be cancelled only by an operator, with a reason, and every bidder is notified.
- **Visibility.** When Unit 1 reports a revoked verification or a suspension, all of the seller's listings get `seller_visible = false`. They leave search, cannot be reserved and cannot be bid on; reinstatement restores them. Status is not changed, so nothing about the listing itself is lost (FR-SRC-04, FR-KYC-10, FR-TNS-05).
- **Relisting.** A seller who relists a similar item shortly after a cancelled or disputed sale is flagged for review (FR-TNS-02, Should). "Similar" means the same category with high title similarity (trigram), within 7 days.

6.2 Photos

Listing photos are public, so unlike KYC images they are not encrypted, but they pass through the same checks as Unit 1's pipeline (SEC-INP-04, SEC-DAT-08):

1. the type is identified from the content (JPEG, PNG or WebP) and the size limit CP-33 applied;
2. the image is decoded and rotated upright, and all metadata including EXIF location is removed;
3. it is re-encoded at 1600 pixels on the long side, with a 400-pixel thumbnail (DV-20);
4. it is stored under a random name on the `listing_media` volume, served with a fixed content type and `nosniff`, and cached by the browser.

Removing EXIF matters here: a phone photo of an item taken at home would otherwise publish the seller's home coordinates.

6.3 AI-drafted text

If the seller asks (FR-LST-13, Could), the photos are sent to the vision model behind `VisionDraftPort`, and the returned title and description are placed in the editor as a suggestion. Nothing is saved until the seller edits or accepts it and saves the draft. The listing records that a draft was used. The feature is switchable off and its failure never blocks listing creation.

7 Search

7.1 Text

PostgreSQL has no Bangla stemmer, so the search vector uses the `simple` configuration. It splits text into words and lower-cases them without language-specific stemming, which works equally for Bangla and English (FR-SRC-01). Title words weigh more than description words. A `pg_trgm` index on the title adds matching on parts of words and tolerates small misspellings, which matters for brand names written in either script.

The query combines the full-text match with a trigram match on the title, ranked by text rank and then recency. The tests confirm that a Bangla word (*mobail* in Bangla script, "mobile") and an English word (*samsung*) both find the same mixed-language listing.

7.2 Filters, distance and sorting

- **Filters** (FR-SRC-02): category, including sub-categories; item kind; sale format; price range, applied to the current price of auctions; condition; and maximum distance.
- **Distance.** A bounding box on the indexed rounded coordinates narrows the candidates, then the haversine distance `listings.distance_km` is computed. The test confirms Dhaka to Chattogram at about 215 km. Distances are shown rounded to the kilometre, from rounded coordinates, so no exact location can be derived (FR-SRC-05). PostGIS was not chosen, because the precision needed is a kilometre and the extra extension is not worth the operational cost.
- **Sorting** (FR-SRC-03): newest, price ascending or descending, distance, and ending soonest (from `auctions_due`).
- **Pagination** is by cursor, using the sort key and the id, so a page never repeats or skips a listing when new ones arrive.
- **What appears.** The view `listings.searchable` contains only active listings of visible

sellers, and among auctions only open ones (FR-SRC-04). Guests use the same endpoint (FR-SRC-05).

7.3 Featured listings and recommendations

Up to DV-21 featured listings that match the query are shown above the results in a strip labelled "Featured". They are not mixed into the organic ranking, and each card is labelled wherever it appears (FR-MON-05). Recommendations (Could, FR-SRC-06) come from the user's recent categories and price band. They appear in their own labelled section, and never include featured placement unless it is labelled as such.

7.4 Performance

For the catalogue of 100,000 listings in NFR-PERF-05, the partial indexes cover only active listings of visible sellers:

- a GIN index on the search vector;
- a trigram index on the title;
- B-tree indexes on category and publication time, on price, and on the coordinates.

A load test with a generated catalogue of that size, and 1,000,000 stored bids for NFR-SCL-03, is part of the test plan.

8 Performance and Scaling

- **Bid latency** (NFR-PERF-02, 500 ms p95). The path is: authentication (one indexed session lookup, Unit 1), identity gate, rate-limit counter in Redis, and one database call that takes the lock. Inside the database the bid decision takes about 0.19 ms (measured over 500 bids). At the reference load of 20 bids per second on one auction, lock waits are therefore short; DV-18 caps them at 1 second.
- **Propagation** (NFR-PERF-03, 1 s p95). One message per committed change goes to the room through the Redis adapter, and each instance forwards it to its own sockets. The 200 viewers of the reference load are spread across instances.
- **Several instances** (NFR-SCL-01, NFR-SCL-02). No instance holds auction state: the database decides and Redis only carries messages. Any instance can serve any bid, and the closer runs safely on all instances at once (`SKIP LOCKED`).
- **Restart** (NFR-REL-01). The authoritative end time and the auction state are in PostgreSQL. Restarting the backend or Redis loses only connections, which reconnect and re-fetch the state.
- **Load test** (to run before the demonstration). A k6 script reproduces NFR-PERF-01: 500 signed-in users, 200 viewers and 20 bids per second on one auction for 60 seconds, 50 simultaneous purchases of a last unit, and search over 100,000 listings. It records p95 latencies and then runs the correctness checks of Appendix B.1 on the result (NFR-PERF-08).

9 Security Design

- **Authorisation.** Listing changes are allowed only for the seller and only in the states permitted. Bids require `@Verified(bid)`. Staff actions are limited to those listed in the API and each needs a reason and is audited (SEC-ACC-01 to SEC-ACC-03, FR-OPS-02).
- **Hidden data.** Maximums, the reserve amount, bidder identities and exact coordinates are never in an API response or a live message (SEC-WEB-05). The public history uses

per-auction aliases. A bidder sees their own maximum through `/my-status`. Operators see maximums only while a fraud flag is open, and each view is audited (FR-AUC-08).

- **Abuse limits.** Bids are limited to 10 per minute per user per auction, listing creation to 10 per day, and search to 60 per minute per address (CP-35, SEC-ABU-01). The listing limit CP-28 is applied under an advisory lock.
- **Content.** Listing text is plain text, encoded on output (SEC-INP-03). Photos are re-encoded without metadata (Section 6.2). Prohibited categories cannot be chosen, and screening terms hold listings for review.
- **Integrity.** Bids are binding and cannot be edited or deleted. History tables are append-only. The application role cannot update them or delete bids (tested).

10 Parameters

SRS parameters used: CP-07 (durations); CP-08 (increment); CP-09 (anti-sniping); CP-10, CP-11 (payment and offer windows, applied by Unit 2 and here); CP-12 (strikes); CP-28 (listing limit); CP-33 (photos); CP-35 (rate limits); CP-37 (featured periods, display only).

Design values introduced (continuing DV-1 to DV-16):

ID	Value	Default	Reason
DV-17	Close grace after `end_at`	2 seconds	Must exceed DV-18 (Section 5.2).
DV-18	Maximum wait for an auction's lock	1 second	Bounds latency; a timed-out bid is answered "try again".
DV-19	Guest polling interval for auction state	5 seconds	Guests have no live channel.
DV-20	Listing photo sizes	1600 px; thumbnail 400 px	Legible on phones; small downloads.
DV-21	Featured listings shown above results	3	Visible, without crowding organic results.
DV-22	Relisting similarity window	7 days, title trigram similarity ≥ 0.6	FR-TNS-02.

11 Testing

Level	Cases
Database (done)	67 checks in `03_listings_auctions_tests.sql`, each required to fail on its intended rule. They cover the listing rules; stock; increments; the worked bid example; ties; reserve lifting; anti-sniping; refusal after the end; strikes; bids being immutable; void and recompute; closing with grace; second chance; Bangla and English search; distance; visibility, including a hidden seller's auction refusing bids; and grants. The script is rerunnable.
Concurrency (done)	Four runs of 300 rising bids from 20 parallel connections. In each run lock order differed from receipt order for 3 to 10 bids, and every live decision equalled its replay. 50 buyers raced for 2 units: exactly 2 reservations (Appendix B.1).
Unit	BidService gates (unverified, suspended, rate-limited); message sequence handling on the client; countdown offset; cursor pagination stability; screening matches in both scripts.

Level	Cases
Integration	Close plus order creation is atomic (order failure rolls back the award). An outbid notice reaches only the previous leader. A reconnecting client re-fetches state. Listings are hidden on suspension and restored. Second-chance chain to exhaustion.
Load	The k6 script of the performance section, with the correctness checks afterwards (AC-1, AC-2).

12 Notes for Integration

I-14 Unit 2. `ListingsPort` is implemented by `listings.reserve`, `release` and `mark_sold`. On an auction order being paid, the listing moves to `sold` through `markSold`. A refund after payment does not return stock automatically; the seller relists.

I-15 Unit 5 adds its game-account details table with a foreign key to `listings.listings` (allowed by DC-5), and approves listings through `ListingsCommands`.

I-16 Unit 6 delivers `auctions.bid.outbid` and the ending-soon notices (FR-NTF-02). Ending-soon notices are driven by `auctions_due`.

I-17 Unit 7 hosts the queues for listings in review, flags and voids, using this unit's staff endpoints.

I-18 SRS clarifications proposed:

- the reserve-lifting rule of Section 5.1 (FR-AUC-09);
- guests see auction state by polling (IF-COM-02);
- an extension caused by a voided bid is kept (FR-AUC-10);
- add DV-17 to DV-22 to the SRS parameter table.

A Traceability

Requirement	Design element	Section
FR-LST-01	Draft fields; kind and format fixed after publication (trigger)	6.1, 4
FR-LST-02	Two photos and defect photo or declaration (trigger)	6.1, 4
FR-LST-03	Parcel classes required for courier (constraint)	4
FR-LST-04	Fixed price needs price and quantity	4
FR-LST-05	Auction start, reserve $\geq$ start, duration CP-07, quantity 1	6.1, 4
FR-LST-06	Rounded coordinates; district shown only	6.1
FR-LST-07	`locked_at`; content lock trigger; clarifications	6.1
FR-LST-08	Cancellation rules; operator cancel with bids	6.1, 3.1
FR-LST-09	Listing limit under advisory lock; Pro status from Unit 2	6.1
FR-LST-10	Prohibited categories refused by trigger	4
FR-LST-11	ScreeningService; `pending_review`	6.1
FR-LST-12	Status machine trigger; `status_history`	6.1
FR-LST-13	`VisionDraftPort`; never saved without the seller	6.3
FR-SRC-01	`simple` text search plus trigrams; Bangla and English tested	7
FR-SRC-02	Filters; bounding box and haversine distance	7
FR-SRC-03	Sorts; cursor pagination	7
FR-SRC-04	`listings.searchable`; `seller_visible`	7, 6.1
FR-SRC-05	Public search; rounded distances	7
FR-SRC-06	Labelled recommendations	7.3
FR-AUC-01	`end_at` in PostgreSQL; client offset from `serverTime`	3.2, 5.2
FR-AUC-02	BidService gates; `place_bid` checks; minimum	5.3, 5.1
FR-AUC-03	Row lock in `place_bid`; processing order	2.3, 5.2
FR-AUC-04	Receipt time `now()`; refusal at or after `end_at`	5.2
FR-AUC-05	Extension to receipt + CP-09	5.2
FR-AUC-06	`auctions.resolve` proxy rules	5.1

Requirement	Design element	Section
FR-AUC-07	Ties to the earlier maximum; price rule checked	5.1
FR-AUC-08	Maximums never returned; flag-gated, audited staff view	5.6, 3.1
FR-AUC-09	Reserve lifting; unsold when not met	5.1, 5.5
FR-AUC-10	Immutable bids; void and replay in processing order	5.4
FR-AUC-11	`auction.state` with sequence numbers	3.2
FR-AUC-12	`auction.outbid` to the previous leader	3.2, 5.3
FR-AUC-13	Aliases; history from replay	5.1, 4
FR-AUC-14	Close plus `createAuctionOrder` in one transaction	5.5
FR-AUC-15	`next_second_chance`; offers; exhaustion to unsold	5.5
FR-AUC-16	`record_strike`; restrictions checked in `place_bid`	5.5
FR-AUC-17	Bid rate limit before the lock; proxy answers not limited	5.3
FR-AUC-18	ShillDetector flags; no automatic blocking	5.6
FR-FIX-02	`listings.reserve`: one conditional update	4, B.1
FR-FIX-04	`mark_sold`; "currently reserved" in the view	4
FR-FIX-05	Definite answer per request; race test 2 of 50	B.1
FR-MON-05	Featured strip, always labelled	7.3
FR-KYC-10	Listings hidden on revocation	6.1
FR-TNS-05	Listings hidden on suspension	6.1
FR-TNS-02	Relisting similarity flag	6.1
FR-TNS-08	Linked-account signal in shill scoring	5.6
SEC-ABU-01	Bid, listing and search limits	3.1, 5.3
SEC-WEB-05	Hidden fields never returned	3.1
SEC-DAT-08	EXIF removed from listing photos	6.2
SEC-INP-04	Photo checks and re-encoding	6.2
SEC-SES-07	Live channel on Unit 1's authenticated socket	3.2
IF-UI-03	Listing, search and auction endpoints and live state	3.1, 3.2
IF-COM-02	Re-fetch on reconnect; guests poll	3.2
NFR-PERF-01	Load-test plan at the reference load	3.1
NFR-PERF-02	One database call per bid (0.19 ms measured)	2.3
NFR-PERF-03	One message per change through the Redis adapter	3.2
NFR-PERF-05	Partial GIN, trigram and B-tree indexes	7
NFR-PERF-08	Correctness checks after concurrency tests	B.1
NFR-REL-01	State in PostgreSQL; Redis only carries messages	2.3
NFR-REL-03	Closer idempotent with SKIP LOCKED	5.5
NFR-SCL-01	Socket.IO Redis adapter	3.2
NFR-SCL-02	No instance-local state	2.3
NFR-SCL-03	Load test with 100,000 listings and 1,000,000 bids	7
NFR-USE-01	Rules link on the auction page; refusal codes explain	3.1
DR-03	`snapshot` for orders	3.1
DR-04	`auctions.bids` with maximums and receipt time	4
DR-10	Append-only histories; immutable bids	4

B Database Schema and Tests

B.1 Test results

Loaded after the Unit 1 and Unit 2 schemas in PostgreSQL 16, using PostgreSQL 15 features only. Every check passed. The concurrency results follow the script output.

```
PASS  FR-LST-10 a prohibited category cannot be used [is prohibited]
PASS  category must match the item kind [does not match item kind]
PASS  a new listing starts as a draft [starts as draft]
PASS  FR-LST-05 an auction has quantity 1 [auction_single_unit]
PASS  FR-LST-03 courier needs weight and size classes [courier_needs_parcel]
PASS  a game account has no physical fields [game_fields]
PASS  FR-LST-02 publishing needs at least two photos [at least two photos]
PASS  FR-LST-02 a defect photo or a no-defects declaration is required [declare that there are none]
```

```
PASS  a listing with two photos, one showing defects, is published
PASS  FR-LST-01 sale format is fixed after publication [fixed after publication]
PASS  reserve 2 of 3
PASS  reserving 2 more of the 1 left is refused
PASS  reserve the last unit
PASS  searchable view shows the listing as currently reserved
PASS  after a release and a sale: 1 available, 0 reserved, 2 sold, still active
PASS  FR-FIX-04 sold when nothing is available or reserved
PASS  stock must always add up [stock_adds_up]
PASS  FR-LST-07 content is locked after the first order [locked after the first bid or order]
PASS  FR-LST-12 sold is terminal [is not permitted]
PASS  clarifications are append-only [append-only]
PASS  increment on BDT 10,000 is 2% = BDT 200
PASS  increment on BDT 1,000 is the BDT 50 minimum
PASS  2% is rounded up to a whole taka (BDT 12,345.67 -> BDT 247)
PASS  first bid leads at the starting price, whatever the maximum
PASS  FR-AUC-02 a bid below price + increment is refused [below_minimum]
PASS  FR-AUC-06 the leader's proxy answers a lower maximum: price = 1,200 + 50
PASS  FR-AUC-07 equal maximums: the earlier one wins, at that maximum
PASS  leader is still the first bidder after the tie
PASS  a higher maximum takes the lead at the old maximum + increment (1,500 + 50)
PASS  a leader can only raise their maximum [max_not_higher]
PASS  raising one's own maximum does not raise the price
PASS  FR-AUC-02 the seller cannot bid [seller_cannot_bid]
PASS  FR-AUC-13 consistent masks: bidders are 1, 2, 3 in order of first bid
PASS  FR-AUC-08 the public history never shows the leader's maximum (250,000 or 200,000)
PASS  recorded live decisions equal a replay of the same bids
PASS  a maximum above the reserve lifts the price to the reserve
PASS  "reserve not met" is cleared
PASS  below the reserve: price stays at the start and "reserve not met" shows
PASS  a bid in the last 2 minutes extends the end to receipt + 2 minutes
PASS  the end time only ever moves later
PASS  FR-AUC-04 a bid received at or after the end is refused [auction_ended]
PASS  one strike does not restrict
PASS  the second strike in 90 days restricts bidding for 30 days
PASS  a restricted bidder is refused [bidder_restricted]
PASS  a bid cannot be changed [can only be voided]
PASS  a bid cannot be deleted [cannot be deleted]
PASS  a bid can be voided only once [only be voided, once]
PASS  FR-AUC-10 after the void the auction is as if those bids never happened
PASS  not closed before end + grace
PASS  the due auction closes as awarded
PASS  awarded to the leader at the current price
PASS  FR-AUC-09 reserve not met closes unsold
PASS  closing again changes nothing
PASS  an award below the reserve is impossible [award_meets_reserve]
PASS  the next eligible bidder is Babu at BDT 1,500
PASS  no further eligible bidder after Babu has been offered
PASS  only one open offer per auction [offer_one_open]
PASS  FR-SRC-01 a Bangla word finds the listing
PASS  FR-SRC-01 an English word finds the same listing
PASS  a part of a word matches through trigrams (galax)
PASS  FR-SRC-02 Dhaka to Chattogram is about 215 km (rounded coordinates)
PASS  closed auctions and sold listings are not searchable
PASS  FR-SRC-04 listings of a suspended or unverified seller disappear from search
PASS  FR-TNS-05 a hidden seller's auction takes no bids [listing not available]
PASS  FR-FIX-05 and FR-SRC-04 a hidden seller's stock cannot be reserved
PASS  application role cannot rewrite listing history [permission denied]
PASS  application role cannot delete bids [permission denied]

Concurrency tests on the final schema (separate connections, one transaction per bid):
  A. Four runs, each 20 bidders x 15 rising bids on one auction in parallel, with random
     delays so that lock order differs from receipt order:
       accepted 32 / 36 / 34 / 35; processed out of receipt order 3 / 10 / 8 / 9;
       in every run: every recorded decision equals the replay; the price never decreased;
       the leader holds the highest maximum; final price = min(top maximum,
       second maximum + increment).
  B. 50 buyers reserving the last 2 units at once: 2 granted, 48 refused.
  Found and fixed while testing: an earlier replay ordered bids by receipt time; under
  contention it disagreed with the live decisions on 11 of 31 bids.
  Timing: place_bid takes about 0.19 ms per call inside the database (500 accepted bids,
  one session, excluding network and commit).
```

B.2 Schema

```
-- ====================================================================
-- TrustBazaar - Unit 3: Auction Engine; Listings, Search & Fixed-Price
-- PostgreSQL 15 schema.  Design document: docs/design/03_listings_auctions.pdf
-- Requirements: SRS v2.0 FR-LST-01..13, FR-SRC-01..06, FR-AUC-01..18,
--   FR-FIX-02/04/05 (stock side), FR-MON-05 (featured display),
--   FR-TNS-02/08 (flags), NFR-PERF-02/03/05, NFR-REL-01, NFR-SCL-01..03.
-- Depends on: 01_identity_access.sql (identity.users, staff, forbid_change,
--   audit.actor_type). Later units (orders, game accounts) are referenced
--   by plain UUIDs (DC-5).
-- The bid engine runs inside the database (auctions.place_bid), so that the
-- critical section - read state, decide, write - happens under one row lock
-- in one round trip. The service layer calls it after its own checks.
-- ====================================================================

CREATE EXTENSION IF NOT EXISTS pg_trgm;      -- substring and fuzzy matching (Bangla and English)

CREATE SCHEMA IF NOT EXISTS listings;
CREATE SCHEMA IF NOT EXISTS auctions;

-- ------------------------------------------------------------------ types
CREATE TYPE listings.item_kind      AS ENUM ('physical', 'game_account');
CREATE TYPE listings.sale_format    AS ENUM ('fixed_price', 'auction');
CREATE TYPE listings.item_condition AS ENUM ('new', 'like_new', 'used', 'for_parts');
CREATE TYPE listings.status         AS ENUM
  ('draft', 'pending_review', 'active', 'sold', 'ended_unsold', 'cancelled', 'removed');   -- FR-LST-12
CREATE TYPE listings.fulfilment_options AS ENUM ('pickup_only', 'courier_only', 'pickup_or_courier');
CREATE TYPE listings.weight_class   AS ENUM ('up_to_1kg', 'up_to_2kg', 'up_to_5kg', 'up_to_10kg', 'over_10kg'
    );
CREATE TYPE listings.size_class     AS ENUM ('small', 'medium', 'large', 'oversize');
CREATE TYPE auctions.auction_status AS ENUM ('open', 'awarded', 'unsold', 'cancelled');
CREATE TYPE auctions.bid_outcome    AS ENUM ('leading', 'outbid', 'raised_own_max');
CREATE TYPE auctions.offer_status   AS ENUM ('offered', 'accepted', 'declined', 'expired');

-- ====================================================================
-- Categories (FR-LST-10)
-- ====================================================================
CREATE TABLE listings.categories (
  id          smallint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  parent_id   smallint REFERENCES listings.categories(id),
  slug        varchar(60) NOT NULL UNIQUE CHECK (slug ~ '^[a-z0-9-]+$'),
  name_en     varchar(80) NOT NULL,
  name_bn     varchar(80) NOT NULL,
  item_kind   listings.item_kind NOT NULL,
  prohibited  boolean NOT NULL DEFAULT false,      -- shown in the policy, never selectable
  sort_order  smallint NOT NULL DEFAULT 0
);

CREATE TABLE listings.screening_terms (                                        -- FR-LST-11
  id        integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  term      varchar(80) NOT NULL UNIQUE,            -- lower case; Bangla or English
  severity  varchar(10) NOT NULL CHECK (severity IN ('prohibited', 'high_risk')),
  added_by  uuid REFERENCES identity.staff_accounts(id),
  added_at  timestamptz NOT NULL DEFAULT now()
);

-- ====================================================================
-- Listings (FR-LST-01..12, FR-FIX-04/05, FR-SRC-04)
-- ====================================================================
CREATE TABLE listings.listings (
  id                  uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  seller_id           uuid NOT NULL REFERENCES identity.users(id),
  item_kind           listings.item_kind NOT NULL,
  sale_format         listings.sale_format NOT NULL,
  category_id         smallint NOT NULL REFERENCES listings.categories(id),
  title               varchar(120) NOT NULL CHECK (length(btrim(title)) >= 5),
  description         varchar(5000) NOT NULL CHECK (length(btrim(description)) >= 20),
  condition           listings.item_condition,
  no_visible_defects  boolean,                     -- the declaration of FR-LST-02
  weight_class        listings.weight_class,
  size_class          listings.size_class,
  fulfilment_options  listings.fulfilment_options,
  area_district       varchar(40),
```

```
  approx_lat           numeric(6,3) CHECK (approx_lat BETWEEN 20.0 AND 27.0),
  approx_lng           numeric(6,3) CHECK (approx_lng BETWEEN 88.0 AND 93.0),
  price                bigint CHECK (price > 0),   -- fixed price, poisha
  quantity_total       integer NOT NULL DEFAULT 1 CHECK (quantity_total >= 1),
  quantity_available   integer NOT NULL DEFAULT 1,
  quantity_reserved    integer NOT NULL DEFAULT 0,
  quantity_sold        integer NOT NULL DEFAULT 0,
  status               listings.status NOT NULL DEFAULT 'draft',
  locked_at            timestamptz,               -- first bid or first order (FR-LST-07)
  seller_visible       boolean NOT NULL DEFAULT true,   -- false while the seller is unverified or suspended
  featured_until       timestamptz,               -- set from payments.featured.* events (FR-MON-05)
  ai_draft_used        boolean NOT NULL DEFAULT false,  -- FR-LST-13 (seller confirmed the text)
  published_at         timestamptz,
  created_at           timestamptz NOT NULL DEFAULT now(),
  updated_at           timestamptz NOT NULL DEFAULT now(),
  search_tsv           tsvector GENERATED ALWAYS AS
                         (setweight(to_tsvector('simple', title), 'A') ||
                          setweight(to_tsvector('simple', description), 'B')) STORED,
  -- stock: every unit is available, reserved by an unpaid order, or sold (FR-FIX-02/05)
  CONSTRAINT stock_non_negative CHECK (quantity_available >= 0 AND quantity_reserved >= 0 AND quantity_sold
     >= 0),
  CONSTRAINT stock_adds_up CHECK (quantity_available + quantity_reserved + quantity_sold = quantity_total),
  CONSTRAINT auction_single_unit CHECK (sale_format = 'fixed_price' OR quantity_total = 1),
  CONSTRAINT fixed_needs_price CHECK ((sale_format = 'fixed_price') = (price IS NOT NULL)),
  -- physical items carry condition, defect declaration, parcel classes, fulfilment and location
  CONSTRAINT physical_fields CHECK (item_kind <> 'physical' OR
     (condition IS NOT NULL AND no_visible_defects IS NOT NULL AND fulfilment_options IS NOT NULL
      AND approx_lat IS NOT NULL AND area_district IS NOT NULL)),
  CONSTRAINT courier_needs_parcel CHECK (fulfilment_options IS NULL OR fulfilment_options = 'pickup_only'
                                      OR (weight_class IS NOT NULL AND size_class IS NOT NULL)),
  CONSTRAINT game_fields CHECK (item_kind <> 'game_account' OR
     (condition IS NULL AND fulfilment_options IS NULL AND weight_class IS NULL AND quantity_total = 1)),
  CONSTRAINT location_pair CHECK ((approx_lat IS NULL) = (approx_lng IS NULL)),
  CONSTRAINT published_time CHECK (status IN ('draft', 'pending_review') OR published_at IS NOT NULL
                                   OR status IN ('cancelled', 'removed'))
);
CREATE INDEX listings_seller   ON listings.listings (seller_id, status);
CREATE INDEX listings_search   ON listings.listings USING gin (search_tsv) WHERE status = 'active';
CREATE INDEX listings_title_trgm ON listings.listings USING gin (title gin_trgm_ops) WHERE status = 'active';
CREATE INDEX listings_browse   ON listings.listings (category_id, published_at DESC) WHERE status = 'active'
     AND seller_visible;
CREATE INDEX listings_price    ON listings.listings (price) WHERE status = 'active' AND seller_visible;
CREATE INDEX listings_geo      ON listings.listings (approx_lat, approx_lng) WHERE status = 'active' AND
     seller_visible;
CREATE INDEX listings_featured ON listings.listings (featured_until) WHERE featured_until IS NOT NULL;

-- edits after the first bid or order, and status transitions (FR-LST-01, 07, 10, 12)
CREATE FUNCTION listings.check_listing_change() RETURNS trigger LANGUAGE plpgsql AS $$
DECLARE prohibited boolean; cat_kind listings.item_kind;
BEGIN
  SELECT c.prohibited, c.item_kind INTO prohibited, cat_kind FROM listings.categories c WHERE c.id = NEW.
     category_id;
  IF prohibited THEN
    RAISE EXCEPTION 'category␣%␣is␣prohibited', NEW.category_id USING ERRCODE = 'check_violation';
  END IF;
  IF cat_kind <> NEW.item_kind THEN
    RAISE EXCEPTION 'category␣does␣not␣match␣item␣kind' USING ERRCODE = 'check_violation';
  END IF;
  IF TG_OP = 'UPDATE' THEN
    IF NEW.item_kind <> OLD.item_kind OR NEW.sale_format <> OLD.sale_format THEN
      IF OLD.status NOT IN ('draft', 'pending_review') THEN
        RAISE EXCEPTION 'item␣kind␣and␣sale␣format␣are␣fixed␣after␣publication' USING ERRCODE = '
     check_violation';
      END IF;
    END IF;
    IF OLD.locked_at IS NOT NULL AND (NEW.title, NEW.description, NEW.price, NEW.quantity_total, NEW.
     category_id, NEW.condition)
         IS DISTINCT FROM (OLD.title, OLD.description, OLD.price, OLD.quantity_total, OLD.category_id, OLD.
     condition) THEN
      RAISE EXCEPTION 'listing␣content␣is␣locked␣after␣the␣first␣bid␣or␣order;␣add␣a␣clarification␣instead'
        USING ERRCODE = 'check_violation';
    END IF;
    IF NEW.status IS DISTINCT FROM OLD.status AND NOT ((OLD.status, NEW.status) IN (
         ('draft'::listings.status, 'pending_review'::listings.status), ('draft', 'active'), ('draft', '
     cancelled'),
```

```
        ('pending_review', 'active'), ('pending_review', 'draft'), ('pending_review', 'removed'),
        ('active', 'sold'), ('active', 'ended_unsold'), ('active', 'cancelled'), ('active', 'removed')))
     THEN
      RAISE EXCEPTION 'listing status % -> % is not permitted', OLD.status, NEW.status USING ERRCODE = '
     check_violation';
    END IF;
    NEW.updated_at := now();
  ELSIF NEW.status NOT IN ('draft', 'pending_review') THEN
    RAISE EXCEPTION 'a new listing starts as draft or pending_review' USING ERRCODE = 'check_violation';
  END IF;
  RETURN NEW;
END $$;
CREATE TRIGGER listing_change BEFORE INSERT OR UPDATE ON listings.listings
  FOR EACH ROW EXECUTE FUNCTION listings.check_listing_change();

CREATE TABLE listings.listing_photos (                                        -- FR-LST-02
  id             uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  listing_id     uuid NOT NULL REFERENCES listings.listings(id),
  position       smallint NOT NULL CHECK (position BETWEEN 1 AND 10),         -- CP-33
  storage_key    varchar(80) NOT NULL UNIQUE,
  width          smallint NOT NULL CHECK (width BETWEEN 1 AND 4000),
  height         smallint NOT NULL CHECK (height BETWEEN 1 AND 4000),
  shows_defects  boolean NOT NULL DEFAULT false,
  created_at     timestamptz NOT NULL DEFAULT now(),
  removed_at     timestamptz,
  UNIQUE (listing_id, position)
);

-- a physical listing is published only with two photos and a defect photo or a no-defects declaration
CREATE FUNCTION listings.check_publishable() RETURNS trigger LANGUAGE plpgsql AS $$
DECLARE n int; defect_photo boolean;
BEGIN
  IF NEW.status IN ('pending_review', 'active') AND OLD.status = 'draft' AND NEW.item_kind = 'physical' THEN
    SELECT count(*), coalesce(bool_or(shows_defects), false) INTO n, defect_photo
      FROM listings.listing_photos WHERE listing_id = NEW.id AND removed_at IS NULL;
    IF n < 2 THEN
      RAISE EXCEPTION 'a physical listing needs at least two photos' USING ERRCODE = 'check_violation';
    END IF;
    IF NOT defect_photo AND NOT NEW.no_visible_defects THEN
      RAISE EXCEPTION 'mark a photo as showing defects or declare that there are none' USING ERRCODE = '
     check_violation';
    END IF;
  END IF;
  RETURN NEW;
END $$;
CREATE TRIGGER listing_publishable BEFORE UPDATE OF status ON listings.listings
  FOR EACH ROW EXECUTE FUNCTION listings.check_publishable();

CREATE TABLE listings.clarifications (                                        -- FR-LST-07, append-only
  id          bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  listing_id  uuid NOT NULL REFERENCES listings.listings(id),
  body        varchar(1000) NOT NULL,
  created_at  timestamptz NOT NULL DEFAULT now()
);
CREATE TRIGGER clarifications_append_only BEFORE UPDATE OR DELETE ON listings.clarifications
  FOR EACH ROW EXECUTE FUNCTION identity.forbid_change();

CREATE TABLE listings.status_history (                                        -- FR-LST-12, append-only
  id          bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  listing_id  uuid NOT NULL REFERENCES listings.listings(id),
  from_status listings.status,
  to_status   listings.status NOT NULL,
  actor_type  audit.actor_type NOT NULL,
  actor_id    uuid,
  reason      text NOT NULL,
  at          timestamptz NOT NULL DEFAULT now()
);
CREATE TRIGGER listing_history_append_only BEFORE UPDATE OR DELETE ON listings.status_history
  FOR EACH ROW EXECUTE FUNCTION identity.forbid_change();

-- ------------------------------------------------------------------ stock port (DC-13): single statements
-- reserve: returns true if reserved; false means "not available" (FR-FIX-05)
CREATE FUNCTION listings.reserve(p_listing uuid, p_qty int) RETURNS boolean LANGUAGE plpgsql AS $$
BEGIN
  UPDATE listings.listings
```

```
     SET quantity_available = quantity_available - p_qty,
         quantity_reserved  = quantity_reserved + p_qty,
         locked_at          = coalesce(locked_at, now())
   WHERE id = p_listing AND status = 'active' AND seller_visible
     AND sale_format = 'fixed_price' AND quantity_available >= p_qty;
  RETURN FOUND;
END $$;

CREATE FUNCTION listings.release(p_listing uuid, p_qty int) RETURNS void LANGUAGE plpgsql AS $$
BEGIN
  UPDATE listings.listings
     SET quantity_available = quantity_available + p_qty,
         quantity_reserved  = quantity_reserved - p_qty
   WHERE id = p_listing;
END $$;

-- on orders.order.paid: reserved -> sold; the listing is sold when nothing is left (FR-FIX-04)
CREATE FUNCTION listings.mark_sold(p_listing uuid, p_qty int) RETURNS void LANGUAGE plpgsql AS $$
BEGIN
  UPDATE listings.listings
     SET quantity_reserved = quantity_reserved - p_qty,
         quantity_sold     = quantity_sold + p_qty
   WHERE id = p_listing;
  UPDATE listings.listings SET status = 'sold'
   WHERE id = p_listing AND status = 'active' AND quantity_available = 0 AND quantity_reserved = 0;
END $$;

-- ---------------------------------------------------------------- distance (FR-SRC-02/05)
-- great-circle distance in km (haversine); inputs are the rounded coordinates
CREATE FUNCTION listings.distance_km(lat1 numeric, lng1 numeric, lat2 numeric, lng2 numeric)
RETURNS double precision LANGUAGE sql IMMUTABLE PARALLEL SAFE AS $$
  SELECT 2 * 6371.0 * asin(sqrt(
      power(sin(radians((lat2 - lat1)::float8) / 2), 2) +
      cos(radians(lat1::float8)) * cos(radians(lat2::float8)) * power(sin(radians((lng2 - lng1)::float8) / 2)
     , 2)))
$$;

-- ====================================================================
-- Auctions (FR-AUC-01..18)
-- ====================================================================
CREATE TABLE auctions.auctions (
  listing_id        uuid PRIMARY KEY REFERENCES listings.listings(id),
  seller_id         uuid NOT NULL REFERENCES identity.users(id),
  start_price       bigint NOT NULL CHECK (start_price > 0),
  reserve_price     bigint,                                   -- hidden (FR-AUC-09)
  -- rule snapshot at publication (FR-OPS-08): CP-08 and CP-09
  inc_bps           integer NOT NULL CHECK (inc_bps BETWEEN 1 AND 5000),          -- 200 = 2 %
  inc_min           bigint  NOT NULL CHECK (inc_min > 0),                         -- 5000 = BDT 50
  snipe_window      interval NOT NULL CHECK (snipe_window > interval '0'),
  snipe_extension   interval NOT NULL CHECK (snipe_extension > interval '0'),
  starts_at         timestamptz NOT NULL,
  original_end_at   timestamptz NOT NULL,
  end_at            timestamptz NOT NULL,                     -- authoritative (FR-AUC-01)
  status            auctions.auction_status NOT NULL DEFAULT 'open',
  leader_id         uuid REFERENCES identity.users(id),
  leader_max        bigint,                                   -- hidden proxy maximum (FR-AUC-08)
  current_price     bigint,
  bid_count         integer NOT NULL DEFAULT 0,
  version           bigint NOT NULL DEFAULT 0,                -- broadcast sequence number
  winner_id         uuid REFERENCES identity.users(id),
  final_price       bigint,
  closed_at         timestamptz,
  CONSTRAINT reserve_at_least_start CHECK (reserve_price IS NULL OR reserve_price >= start_price),
  CONSTRAINT end_only_extends CHECK (end_at >= original_end_at AND original_end_at > starts_at),
  CONSTRAINT leader_fields CHECK ((leader_id IS NULL) = (leader_max IS NULL) AND (leader_id IS NULL) = (
     current_price IS NULL)),
  CONSTRAINT price_within_max CHECK (current_price IS NULL OR (current_price >= start_price AND current_price
      <= leader_max)),
  CONSTRAINT seller_not_leader CHECK (leader_id IS NULL OR leader_id <> seller_id),
  CONSTRAINT award_fields CHECK ((status = 'awarded') = (winner_id IS NOT NULL AND final_price IS NOT NULL)),
  CONSTRAINT closed_time CHECK ((status = 'open') = (closed_at IS NULL)),
  CONSTRAINT award_meets_reserve CHECK (status <> 'awarded' OR reserve_price IS NULL OR final_price >=
     reserve_price)
);
```

```
CREATE INDEX auctions_due ON auctions.auctions (end_at) WHERE status = 'open';      -- close job, "ending
    soonest"

CREATE TABLE auctions.bids (                                    -- inputs; bids are binding (FR-AUC-10)
  id            bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  auction_id    uuid NOT NULL REFERENCES auctions.auctions(listing_id),
  bidder_id     uuid NOT NULL REFERENCES identity.users(id),
  max_amount    bigint NOT NULL CHECK (max_amount > 0),         -- the bidder's maximum (a plain bid is its
    own maximum)
  received_at   timestamptz NOT NULL,                           -- server receipt time (FR-AUC-04)
  outcome       auctions.bid_outcome NOT NULL,                  -- as decided when placed
  price_after   bigint NOT NULL,
  leader_after  uuid NOT NULL,
  extended_to   timestamptz,                                    -- set when the bid triggered anti-sniping
  voided_at     timestamptz,
  voided_by     uuid REFERENCES identity.staff_accounts(id),
  void_reason   text,
  CONSTRAINT void_fields CHECK ((voided_at IS NULL) = (voided_by IS NULL) AND (voided_at IS NULL) = (
    void_reason IS NULL))
);
CREATE INDEX bids_auction ON auctions.bids (auction_id, received_at, id);
CREATE INDEX bids_bidder  ON auctions.bids (bidder_id, received_at DESC);
-- bids are never deleted, and only the void fields may change
CREATE FUNCTION auctions.bids_only_void() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
  IF TG_OP = 'DELETE' THEN RAISE EXCEPTION 'bids␣cannot␣be␣deleted' USING ERRCODE = 'insufficient_privilege';
     END IF;
  IF (NEW.auction_id, NEW.bidder_id, NEW.max_amount, NEW.received_at, NEW.outcome, NEW.price_after, NEW.
    leader_after)
     IS DISTINCT FROM (OLD.auction_id, OLD.bidder_id, OLD.max_amount, OLD.received_at, OLD.outcome, OLD.
    price_after, OLD.leader_after)
     OR OLD.voided_at IS NOT NULL THEN
    RAISE EXCEPTION 'a␣bid␣can␣only␣be␣voided,␣once' USING ERRCODE = 'insufficient_privilege';
  END IF;
  RETURN NEW;
END $$;
CREATE TRIGGER bids_immutable BEFORE UPDATE OR DELETE ON auctions.bids
  FOR EACH ROW EXECUTE FUNCTION auctions.bids_only_void();

CREATE TABLE auctions.bidder_aliases (                          -- consistent mask per auction (FR-AUC-13)
  auction_id  uuid NOT NULL REFERENCES auctions.auctions(listing_id),
  bidder_id   uuid NOT NULL REFERENCES identity.users(id),
  alias_no    integer NOT NULL,
  PRIMARY KEY (auction_id, bidder_id),
  UNIQUE (auction_id, alias_no)
);

CREATE TABLE auctions.strikes (                                 -- FR-AUC-16
  id          bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  user_id     uuid NOT NULL REFERENCES identity.users(id),
  auction_id  uuid NOT NULL REFERENCES auctions.auctions(listing_id),
  order_id    uuid NOT NULL,                                    -- Unit 2 order (no FK)
  at          timestamptz NOT NULL DEFAULT now(),
  UNIQUE (order_id)                                             -- one strike per unpaid order
);
CREATE TABLE auctions.bidding_restrictions (
  user_id     uuid PRIMARY KEY REFERENCES identity.users(id),
  until       timestamptz NOT NULL,
  reason      text NOT NULL,
  set_at      timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE auctions.second_chance_offers (                    -- FR-AUC-15
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  auction_id  uuid NOT NULL REFERENCES auctions.auctions(listing_id),
  bidder_id   uuid NOT NULL REFERENCES identity.users(id),
  price       bigint NOT NULL CHECK (price > 0),
  status      auctions.offer_status NOT NULL DEFAULT 'offered',
  offered_at  timestamptz NOT NULL DEFAULT now(),
  expires_at  timestamptz NOT NULL,
  decided_at  timestamptz,
  order_id    uuid,
  CHECK ((status = 'offered') = (decided_at IS NULL)),
  CHECK ((status = 'accepted') = (order_id IS NOT NULL)),
  UNIQUE (auction_id, bidder_id)                                -- a bidder is offered the item at most once
```

```
);
CREATE UNIQUE INDEX offer_one_open ON auctions.second_chance_offers (auction_id) WHERE status = 'offered';

CREATE TABLE auctions.flags (                                    -- FR-AUC-18, FR-TNS-08
  id          bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  auction_id  uuid NOT NULL REFERENCES auctions.auctions(listing_id),
  kind        varchar(40) NOT NULL CHECK (kind IN ('new_account_cluster', 'linked_to_seller', '
     last_second_pattern', 'self_outbid_pattern')),
  score       numeric(4,3) NOT NULL CHECK (score BETWEEN 0 AND 1),
  detail      jsonb NOT NULL,
  created_at  timestamptz NOT NULL DEFAULT now(),
  reviewed_at timestamptz
);

CREATE TABLE auctions.outbox (
  id              bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  event_type      varchar(80) NOT NULL CHECK (event_type ~ '^(listings|auctions)\.[a-z_]+\.[a-z_]+$'),
  aggregate_id    uuid NOT NULL,
  payload         jsonb NOT NULL,
  created_at      timestamptz NOT NULL DEFAULT now(),
  attempts        smallint NOT NULL DEFAULT 0,
  next_attempt_at timestamptz NOT NULL DEFAULT now(),
  dispatched_at   timestamptz
);
CREATE INDEX auctions_outbox_pending ON auctions.outbox (next_attempt_at) WHERE dispatched_at IS NULL;

-- ====================================================================
-- Bid engine (FR-AUC-02..09, FR-AUC-13)
-- ====================================================================
-- minimum increment over a price: greater of inc_bps of the price and inc_min,
-- rounded up to a whole taka (CP-08)
CREATE FUNCTION auctions.increment(p_price bigint, p_bps int, p_min bigint) RETURNS bigint
LANGUAGE sql IMMUTABLE PARALLEL SAFE AS $$
  SELECT greatest(((p_price * p_bps + 999999) / 1000000) * 100, p_min)
$$;

-- Pure decision: given the current state and one new maximum, return the new state.
-- Used by place_bid and by replay, so a recomputation after a void follows exactly the same rules.
-- Raises 'below_minimum' or 'max_not_higher' when the bid is not acceptable.
CREATE FUNCTION auctions.resolve(
    p_start bigint, p_reserve bigint, p_bps int, p_min bigint,
    p_leader uuid, p_leader_max bigint, p_price bigint,
    p_bidder uuid, p_max bigint,
    OUT leader uuid, OUT leader_max bigint, OUT price bigint, OUT outcome auctions.bid_outcome)
LANGUAGE plpgsql IMMUTABLE AS $$
DECLARE required bigint;
BEGIN
  IF p_leader IS NULL THEN                                       -- first bid
    IF p_max < p_start THEN
      RAISE EXCEPTION 'below_minimum: first bid must be at least %', p_start USING ERRCODE = 'P0001';
    END IF;
    leader := p_bidder; leader_max := p_max; price := p_start; outcome := 'leading';
  ELSIF p_bidder = p_leader THEN                                 -- leader raises own maximum
    IF p_max <= p_leader_max THEN
      RAISE EXCEPTION 'max_not_higher: new maximum must exceed your current maximum' USING ERRCODE = 'P0001';
    END IF;
    leader := p_leader; leader_max := p_max; price := p_price; outcome := 'raised_own_max';
  ELSE
    required := p_price + auctions.increment(p_price, p_bps, p_min);
    IF p_max < required THEN
      RAISE EXCEPTION 'below_minimum: bid must be at least %', required USING ERRCODE = 'P0001';
    END IF;
    IF p_max > p_leader_max THEN                                 -- challenger takes the lead
      leader := p_bidder; leader_max := p_max;
      price := least(p_max, p_leader_max + auctions.increment(p_leader_max, p_bps, p_min));
      outcome := 'leading';
    ELSE                                                         -- leader's proxy answers; ties go to the
     earlier maximum
      leader := p_leader; leader_max := p_leader_max;
      price := least(p_leader_max, p_max + auctions.increment(p_max, p_bps, p_min));
      outcome := 'outbid';
    END IF;
  END IF;
  -- reserve: if the leader's maximum reaches the reserve, the price rises to it, so that a bidder
  -- willing to pay the reserve is not left below it by the proxy step (FR-AUC-09)
```

```
  IF p_reserve IS NOT NULL AND price < p_reserve AND leader_max >= p_reserve THEN
    price := p_reserve;
  END IF;
END $$;

-- Place one bid. The caller (BidService) has already checked that the bidder is verified, not
-- suspended, within the rate limit, and that the auction's listing is visible. The transaction
-- must start when the request is received: now() (transaction start) is the receipt time.
CREATE FUNCTION auctions.place_bid(p_auction uuid, p_bidder uuid, p_max bigint,
    OUT bid_id bigint, OUT outcome auctions.bid_outcome, OUT price bigint, OUT leader_alias int,
    OUT end_at timestamptz, OUT extended boolean, OUT previous_leader uuid, OUT version bigint)
LANGUAGE plpgsql AS $$
DECLARE
  a auctions.auctions%ROWTYPE;
  r record;
  received timestamptz := now();
  restricted timestamptz;
BEGIN
  SET LOCAL lock_timeout = '1s';                                    -- bounds waiting; see close grace DV-17
  SELECT * INTO a FROM auctions.auctions WHERE listing_id = p_auction FOR UPDATE;
  IF NOT FOUND THEN RAISE EXCEPTION 'auction_not_found' USING ERRCODE = 'P0001'; END IF;
  IF a.status <> 'open' OR received < a.starts_at THEN
    RAISE EXCEPTION 'auction_not_open' USING ERRCODE = 'P0001';
  END IF;
  -- the listing must be active and its seller visible (not unverified or suspended: FR-SRC-04, FR-TNS-05)
  PERFORM 1 FROM listings.listings WHERE id = p_auction AND status = 'active' AND seller_visible;
  IF NOT FOUND THEN RAISE EXCEPTION 'auction_not_open: listing not available' USING ERRCODE = 'P0001'; END IF
     ;
  IF received >= a.end_at THEN                                      -- decided by receipt time (FR-AUC-04)
    RAISE EXCEPTION 'auction_ended' USING ERRCODE = 'P0001';
  END IF;
  IF p_bidder = a.seller_id THEN
    RAISE EXCEPTION 'seller_cannot_bid' USING ERRCODE = 'P0001';
  END IF;
  SELECT until INTO restricted FROM auctions.bidding_restrictions WHERE user_id = p_bidder;
  IF restricted IS NOT NULL AND restricted > received THEN
    RAISE EXCEPTION 'bidder_restricted until %', restricted USING ERRCODE = 'P0001';
  END IF;

  r := auctions.resolve(a.start_price, a.reserve_price, a.inc_bps, a.inc_min,
                        a.leader_id, a.leader_max, a.current_price, p_bidder, p_max);

  previous_leader := a.leader_id;
  extended := received >= a.end_at - a.snipe_window;                -- anti-sniping (FR-AUC-05)
  end_at := CASE WHEN extended THEN greatest(a.end_at, received + a.snipe_extension) ELSE a.end_at END;

  INSERT INTO auctions.bidder_aliases (auction_id, bidder_id, alias_no)
  SELECT p_auction, p_bidder, coalesce(max(alias_no), 0) + 1 FROM auctions.bidder_aliases WHERE auction_id =
     p_auction
  ON CONFLICT (auction_id, bidder_id) DO NOTHING;

  INSERT INTO auctions.bids (auction_id, bidder_id, max_amount, received_at, outcome, price_after,
     leader_after, extended_to)
  VALUES (p_auction, p_bidder, p_max, received, r.outcome, r.price, r.leader, CASE WHEN extended THEN end_at
     END)
  RETURNING id INTO bid_id;

  UPDATE auctions.auctions
     SET leader_id = r.leader, leader_max = r.leader_max, current_price = r.price,
         bid_count = auctions.auctions.bid_count + 1, end_at = place_bid.end_at,
         version = auctions.auctions.version + 1
   WHERE listing_id = p_auction
  RETURNING auctions.auctions.version INTO version;

  IF a.bid_count = 0 THEN                                           -- first bid locks the listing (FR-LST-07)
    UPDATE listings.listings SET locked_at = coalesce(locked_at, received) WHERE id = p_auction;
  END IF;

  SELECT alias_no INTO leader_alias FROM auctions.bidder_aliases WHERE auction_id = p_auction AND bidder_id =
      r.leader;
  outcome := r.outcome; price := r.price;
END $$;

-- Replay all bids that are not voided, in the order they were processed, through the same rules
-- (FR-AUC-10). Processing order is the bid id order: ids are assigned under the auction's row lock,
```

```
-- so they record the sequence in which bids were actually decided. Receipt time can differ (a bid
-- received earlier may wait longer for the lock), and is used only for the end-time rule.
-- Returns the public bid history: bidder alias, the amount shown, and whether that bid led.
-- A bid that would be rejected on replay (possible only after a void) is shown as not counted.
CREATE FUNCTION auctions.replay(p_auction uuid)
RETURNS TABLE (bid_id bigint, received_at timestamptz, alias_no int, shown_amount bigint,
               is_leading boolean, counted boolean, leader uuid, leader_max bigint, price bigint)
LANGUAGE plpgsql STABLE AS $$
DECLARE
  a auctions.auctions%ROWTYPE;
  b record;
  st record;
  cur_leader uuid; cur_max bigint; cur_price bigint;
BEGIN
  SELECT * INTO a FROM auctions.auctions WHERE listing_id = p_auction;
  FOR b IN SELECT x.id, x.bidder_id, x.max_amount, x.received_at FROM auctions.bids x
           WHERE x.auction_id = p_auction AND x.voided_at IS NULL ORDER BY x.id LOOP
    BEGIN
      st := auctions.resolve(a.start_price, a.reserve_price, a.inc_bps, a.inc_min,
                             cur_leader, cur_max, cur_price, b.bidder_id, b.max_amount);
      cur_leader := st.leader; cur_max := st.leader_max; cur_price := st.price;
      bid_id := b.id; received_at := b.received_at;
      SELECT ba.alias_no INTO alias_no FROM auctions.bidder_aliases ba WHERE ba.auction_id = p_auction AND ba
     .bidder_id = b.bidder_id;
      -- a leading bid shows the price it set; an outbid bid shows the bidder's own amount;
      -- the leader's maximum is never shown (FR-AUC-08, FR-AUC-13)
      is_leading := st.outcome <> 'outbid';
      shown_amount := CASE WHEN st.outcome = 'outbid' THEN b.max_amount ELSE st.price END;
      counted := true; leader := cur_leader; leader_max := cur_max; price := cur_price;
      RETURN NEXT;
    EXCEPTION WHEN raise_exception THEN
      bid_id := b.id; received_at := b.received_at; alias_no := NULL; shown_amount := NULL;
      is_leading := false; counted := false; leader := cur_leader; leader_max := cur_max; price := cur_price;
      RETURN NEXT;
    END;
  END LOOP;
END $$;

-- Recompute the auction after a void (FR-AUC-10). The end time is not shortened: an extension
-- caused by a voided bid stays, because other bidders may have relied on it.
CREATE FUNCTION auctions.recompute(p_auction uuid) RETURNS void LANGUAGE plpgsql AS $$
DECLARE lst record; n int;
BEGIN
  PERFORM 1 FROM auctions.auctions WHERE listing_id = p_auction AND status = 'open' FOR UPDATE;
  IF NOT FOUND THEN RAISE EXCEPTION 'only␣an␣open␣auction␣can␣be␣recomputed'; END IF;
  SELECT count(*) FILTER (WHERE counted) INTO n FROM auctions.replay(p_auction);
  SELECT * INTO lst FROM auctions.replay(p_auction) r ORDER BY r.bid_id DESC LIMIT 1;
  UPDATE auctions.auctions
     SET leader_id = lst.leader, leader_max = lst.leader_max, current_price = lst.price,
         bid_count = coalesce(n, 0), version = version + 1
   WHERE listing_id = p_auction;
END $$;

-- Close one auction if it is due (FR-AUC-09, FR-AUC-14). p_grace (DV-17) must exceed the bid
-- lock_timeout, so that every bid received before end_at has finished before the close decides.
-- Returns the new status; the caller creates the order (Unit 2 OrdersPort) in the same transaction.
CREATE FUNCTION auctions.close_if_due(p_auction uuid, p_grace interval)
RETURNS auctions.auction_status LANGUAGE plpgsql AS $$
DECLARE a auctions.auctions%ROWTYPE;
BEGIN
  SELECT * INTO a FROM auctions.auctions WHERE listing_id = p_auction FOR UPDATE;
  IF a.status <> 'open' OR now() < a.end_at + p_grace THEN
    RETURN a.status;                                          -- not due, or already closed
  END IF;
  IF a.leader_id IS NULL OR (a.reserve_price IS NOT NULL AND a.current_price < a.reserve_price) THEN
    UPDATE auctions.auctions SET status = 'unsold', closed_at = now() WHERE listing_id = p_auction;
    RETURN 'unsold';
  END IF;
  UPDATE auctions.auctions SET status = 'awarded', winner_id = a.leader_id, final_price = a.current_price,
     closed_at = now()
   WHERE listing_id = p_auction;
  RETURN 'awarded';
END $$;

-- Next eligible bidder for a second-chance offer (FR-AUC-15): highest maximum among bidders who
```

```
-- have not won or been offered the item, whose bids are not voided and who are not restricted.
CREATE FUNCTION auctions.next_second_chance(p_auction uuid, OUT bidder uuid, OUT price bigint)
LANGUAGE sql STABLE AS $$
  SELECT b.bidder_id, max(b.max_amount)
    FROM auctions.bids b JOIN auctions.auctions a ON a.listing_id = b.auction_id
   WHERE b.auction_id = p_auction AND b.voided_at IS NULL
     AND b.bidder_id IS DISTINCT FROM a.winner_id
     AND NOT EXISTS (SELECT 1 FROM auctions.second_chance_offers o WHERE o.auction_id = p_auction AND o.
    bidder_id = b.bidder_id)
     AND NOT EXISTS (SELECT 1 FROM auctions.bidding_restrictions r WHERE r.user_id = b.bidder_id AND r.until
    > now())
   GROUP BY b.bidder_id
   ORDER BY max(b.max_amount) DESC, min(b.id)
   LIMIT 1
$$;

-- Record a non-payment strike; restrict bidding on reaching the threshold (FR-AUC-16, CP-12)
CREATE FUNCTION auctions.record_strike(p_user uuid, p_auction uuid, p_order uuid,
                                       p_threshold int, p_window interval, p_ban interval) RETURNS boolean
LANGUAGE plpgsql AS $$
DECLARE n int;
BEGIN
  INSERT INTO auctions.strikes (user_id, auction_id, order_id) VALUES (p_user, p_auction, p_order)
  ON CONFLICT (order_id) DO NOTHING;
  SELECT count(*) INTO n FROM auctions.strikes WHERE user_id = p_user AND at > now() - p_window;
  IF n >= p_threshold THEN
    INSERT INTO auctions.bidding_restrictions (user_id, until, reason)
    VALUES (p_user, now() + p_ban, format('%s non-payment strikes within %s', n, p_window))
    ON CONFLICT (user_id) DO UPDATE SET until = greatest(auctions.bidding_restrictions.until, EXCLUDED.until)
     ,
                                        reason = EXCLUDED.reason, set_at = now();
    RETURN true;
  END IF;
  RETURN false;
END $$;

-- ============================================================================
-- Search view (FR-SRC-01..05): only active listings of visible sellers, open auctions only
-- ============================================================================
CREATE VIEW listings.searchable AS
  SELECT l.id, l.seller_id, l.item_kind, l.sale_format, l.category_id, l.title, l.condition,
         l.area_district, l.approx_lat, l.approx_lng, l.published_at, l.search_tsv,
         CASE WHEN l.sale_format = 'auction' THEN coalesce(a.current_price, a.start_price) ELSE l.price END
    AS display_price,
         a.end_at AS auction_end_at,
         (a.reserve_price IS NOT NULL AND coalesce(a.current_price, 0) < a.reserve_price) AS reserve_not_met,
         (l.featured_until IS NOT NULL AND l.featured_until > now()) AS featured,
         (l.quantity_available = 0 AND l.quantity_reserved > 0) AS currently_reserved
    FROM listings.listings l
    LEFT JOIN auctions.auctions a ON a.listing_id = l.id
   WHERE l.status = 'active' AND l.seller_visible
     AND (l.sale_format = 'fixed_price' OR a.status = 'open');

-- ============================================================================
-- Grants (SEC-INF-03)
-- ============================================================================
GRANT USAGE ON SCHEMA listings, auctions TO tb_app;
GRANT SELECT, INSERT, UPDATE ON ALL TABLES IN SCHEMA listings, auctions TO tb_app;
REVOKE UPDATE ON listings.clarifications, listings.status_history FROM tb_app;
GRANT SELECT ON listings.searchable TO tb_app;
GRANT EXECUTE ON ALL FUNCTIONS IN SCHEMA listings, auctions TO tb_app;
```